Integer Representations

Outline

- Encodings
  - Unsigned and two's complement
- Conversions
  - Signed vs. unsigned
  - Long vs. short
- Suggested reading
  - Chap 2.2

Integral Data Types P51 Figure 2.8

- C supports a variety of integral data types
  - Represent a finite range of integers

C declaration	guaranteed		Typical 32-bit	
	minimum	maximum	minimum	maximum
char	-127	127	-128	127
unsigned char	0	255	0	255
short [int]	-32,767	32,767	-32,768	32,767
unsigned short	0	65,535	0	65,535
int	-32,767	32,767	-2,147,483,648	2,147,483,647
unsigned [int]	0	65,535	0	4,294,967,295
long [int]	-2,147,483,647	2,147,483,647	-2,147,483,648	2,147,483,647
unsigned long	0	0	0	4,294,967,295

Two's Complement

- Binary
  - Bit vector $[x_{w-1}, x_{w-2}, x_{w-3}, \ldots x_0]$
- Using 2's complement to represent integer

Unsigned

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

P52 Eq. (2.1)

Two's Complement

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Sign Bit

P52 Eq. (2.2)

From Two's Complement to Binary

- If nonnegative
  - Nothing changes
- If negative

$$2^{w-1} = \sum_{i=0}^{w-2} 2^i + 1$$

$$2^{w-1} - \sum_{i=0}^{w-2} x_i 2^i = \sum_{i=0}^{w-2} (1 - x_i) 2^i + 1$$

Two's Complement

- Two's Complement

-5	0101 (raw binary)
	1010 (after complement)
	1011 (2's complement)

Two's Complement Encoding Examples

Binary/Hexadecimal Representation for 12345

Binary: 0011 0000 0011 1001

Hex: 3 0 3 9

Binary/Hexadecimal Representation for -12345

Binary: 1100 1111 1100 0111

Hex: C F C 7

P55 Figure 2.10

Weight	12,345		−12,345		53,191	
	Bit	Value	Bit	Value	Bit	Value
1	1	1	1	1	1	1
2	0	0	1	2	1	2
4	0	0	1	4	1	4
8	1	8	0	0	0	0
16	1	16	0	0	0	0
32	1	32	0	0	0	0
64	0	0	1	64	1	64
128	0	0	1	128	1	128
256	0	0	1	256	1	256
512	0	0	1	512	1	512
1,024	0	0	1	1,024	1	1,024
2,048	0	0	1	2,048	1	2,048
4,096	1	4096	0	0	0	0
8,192	1	8192	0	0	0	0
16,384	0	0	1	16,384	1	16,384
±32, 768	0	0	1	−32,768	1	32,768
Total		12,345		−12,345		53,191

Numeric Range

- Unsigned Values
  - Umin=0
  - Umax=2^w-1
- Two's Complement Values
  - Tmin = -2^{w-1}
  - Tmax = $2^{w-1}-1$

Interesting Numbers P53 Figure 2.9

Quantity	Word Size w			
	8	16	32	64
$UMax_w$	0xFF 255	0xFFFF 65,535	0xFFFFFFFF 4,294,967,295	0xFFFFFFFFFFFFFFFF 18,446,744,073,709,551,615
$TMax_w$	0x7F 127	0x7FFF 32,767	0x7FFFFFFF 2,147,483,647	0x7FFFFFFFFFFFFFFF 9,223,372,036,854,775,807
$TMin_w$	0x80 −128	0x8000 −32,768	0x80000000 −2,147,483,648	0x8000000000000000 −9,223,372,036,854,775,808
−1	0xFF	0xFFFF	0xFFFFFFFF	0xFFFFFFFFFFFFFFFF
0	0x00	0x0000	0x00000000	0x0000000000000000

Numeric Range

- Relationship
  - $|TMin| = TMax + 1$
  - $Umax = 2*TMax + 1$
  - -1 has the same bit representation as *Umax*,
    - a string of all 1s
  - Numeric value 0 is represented as
    - a string of all 0s in both representations

X	B2U(*X*)	B2T(*X*)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	–8
1001	9	–7
1010	10	–6
1011	11	–5
1100	12	–4
1101	13	–3
1110	14	–2
1111	15	–1

Unsigned & Signed Numeric Values

- Equivalence
  - Same encodings for nonnegative values
- Uniqueness
  - Every bit pattern represents unique integer value
  - Each representable integer has unique bit encoding

Unsigned & Signed Numeric Values

- $\Rightarrow$ Can Invert Mappings
  - $U2B(x) = B2U^{-1}(x)$
    - Bit pattern for unsigned integer
  - $T2B(x) = B2T^{-1}(x)$
    - Bit pattern for two's comp integer

Alternative representations of signed numbers P54

- One's Complement:
  - The most significant bit has weight $-(2^{w-1}-1)$
- Sign-Magnitude
  - The most significant bit is a sign bit
    - that determines whether the remaining bits should be given negative or positive weight

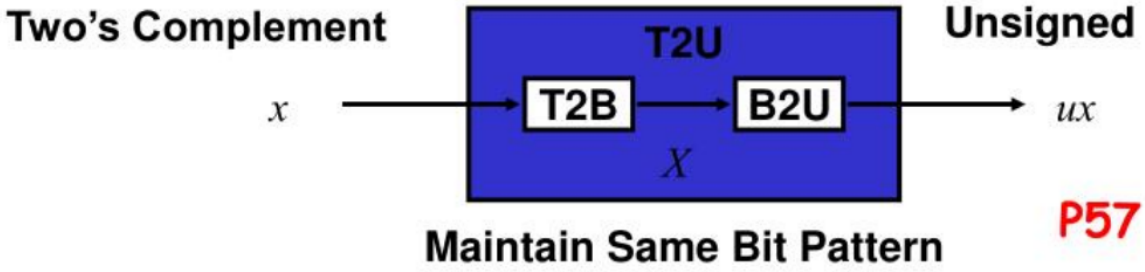
Casting Signed to Unsigned

- C Allows Conversions from Signed to Unsigned

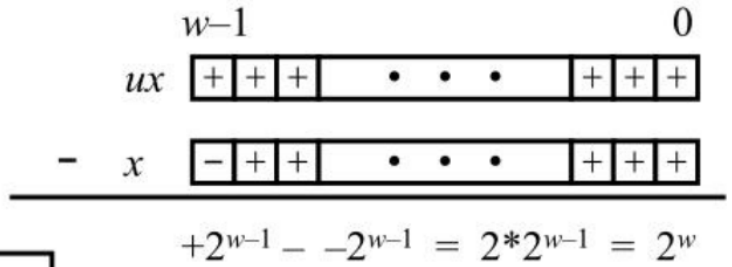
```
short int          x =  12345;
unsigned short int ux = (unsigned short) x;
short int          y  = -12345;
unsigned short int uy = (unsigned short) y;
```

- Resulting Value
  - No change in bit representation
  - Nonnegative values unchanged
    - *ux* = 12345
  - Negative values change into (large) positive values
    - *uy* = 53191

Relation Between 2's Comp. & Unsigned P57



P57 Eq. (2.3)

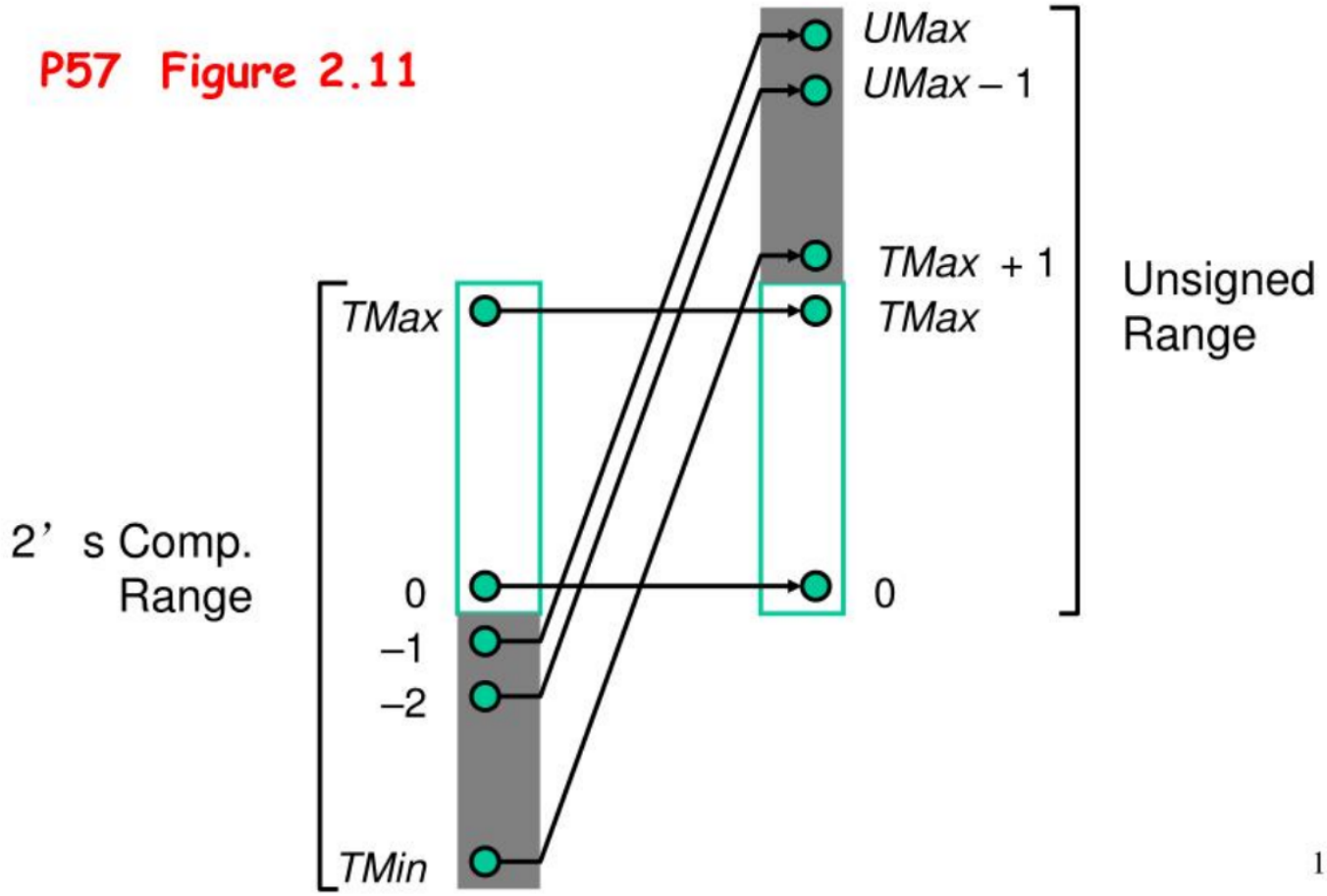


$$ux = \begin{cases} x & x \geq 0 \\ x + 2^w & x < 0 \end{cases}$$

P57 Eq. (2.4)

Conversion between two Representations

P57 Figure 2.11



Signed vs. Unsigned in C

- Constants
  - By default are considered to be signed integers
  - Unsigned if have "U" as suffix
    - 0U, 4294967259U

Signed vs. Unsigned in C P59

- Casting
  - Explicit casting between signed & unsigned same as U2T and T2U
    - int tx, ty;
    - unsigned ux, uy;
    - tx = (int) ux;
    - uy = (unsigned) ty;

Signed vs. Unsigned in C

- Casting
  - Implicit casting also occurs via assignments and procedure calls
    - int tx, ty;
    - unsigned ux, uy;
    - tx = ux; /* Cast to signed */
    - uy = ty; /* Cast to unsigned */

Casting Convention

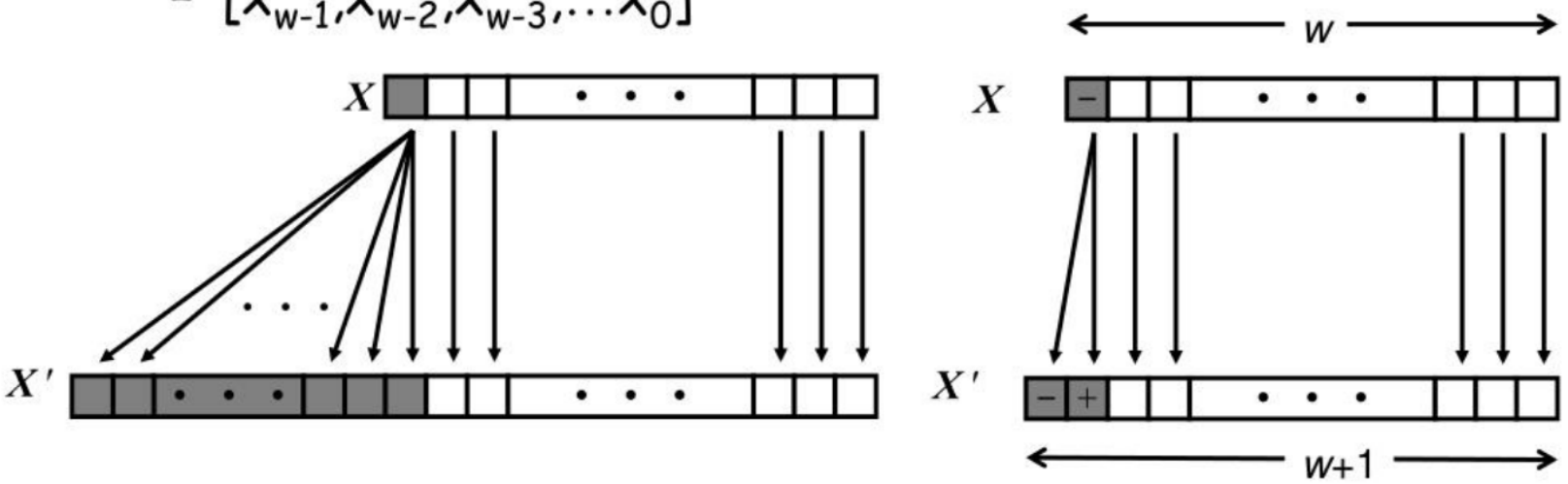
- Expression Evaluation
  - If mix unsigned and signed in single expression
    - signed values implicitly cast to unsigned
  - Including comparison operations <, >, ==, <=, >=
  - Examples for *W* = 32

Casting Convention P60 Figure 2.13

Constant1	Constant2	Relation	Evaluation
0	0U	==	unsigned
-1	0	<	signed
-1	0U	>	unsigned
2147483647	-2147483648	>	signed
2147483647U	-2147483648	<	unsigned
-1	-2	>	signed
(unsigned)-1	-2	>	unsigned

Expanding the Bit Representation P61

- Zero extension
  - Add leading 0s to the representation
- Sign extension
  - $[x_{w-1}, x_{w-2}, x_{w-3}, \ldots x_0]$



Sign Extension Example

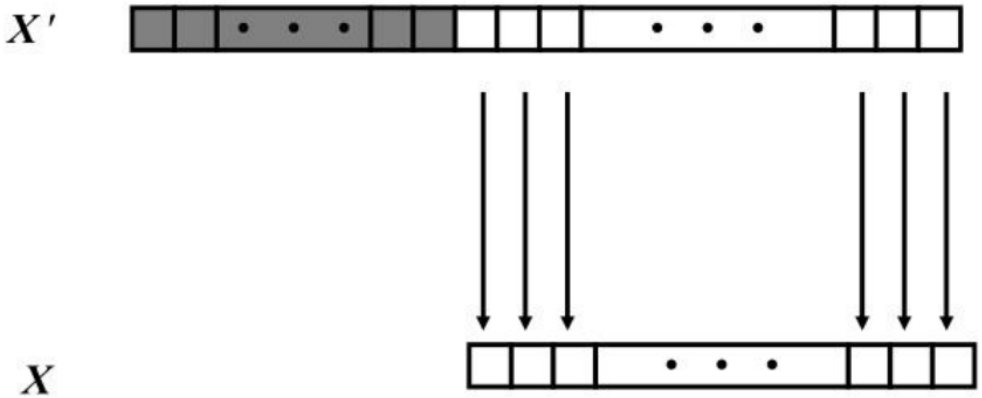
```
short int x =  12345;
int       ix = (int) x;
short int y = -12345;
int       iy = (int) y;
```

	Decimal	Hex	Binary
x	12345	30 39	00110000 00111001
ix	12345	00 00 30 39	00000000 00000000 00110000 00111001
y	-12345	CF C7	11001111 11000111
iy	-12345	FF FF CF C7	11111111 11111111 11001111 11000111

Truncating Numbers P63

```
int       x  = 53191;
short int sx = -12345;
int       y  = -12345;
```

	Decimal	Hex	Binary
x	53191	00 00 CF C7	00000000 00000000 11001111 11000111
sx	-12345	CF C7	11001111 11000111
y	-12345	FF FF CF C7	11111111 11111111 11001111 11000111



Truncating Numbers

- Unsigned Truncating P64 Eq. (2.7)

$$B2U_w([x_w, x_{w-1}, \cdots, x_0]) \bmod 2^k = B2U_k([x_k, x_{k-1}, \cdots x_0])$$

- Signed Truncating P64 Eq. (2.8)

$$B2T_k([x_k, x_{k-1}, \cdots x_0]) = U2T_k(B2U_w([x_w, x_{w-1}, \cdots, x_0]) \bmod 2^k)$$

Advice on Signed vs. Unsigned P65 Practice Problem 2.23 [Solution P115]

Nonintuitive Features

```
unsigned length ;
int i ;
for ( i = 0; i <= length - 1; i++)
    result += a[i] ;
```

Advice on Signed vs. Unsigned

- Collections of bits
  - Bit vectors
  - Masks
- Addresses
- Multiprecision Arithmetic
  - Numbers are represented by arrays of words